

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 4 – Enkapsulasi



NAMA : Ruhfaidah Shirotul Akhiroh

NIM : 103132400019

KELAS : S1SD-05-REG-01

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

1. Java Modifier

Modifier memberi dampak tertentu pada kelas, *interface*, dan variabel. Java Modifier dibagi menjadi kelompok;

- Access modifier berlaku untuk kelas, *method*, dan variabel, meliputi *modifier* *public*, *protected*, *private*, dan tak ada modifier.
- Final modifier berlaku untuk kelas, *method*, dan variabel, meliputi *modifier* *final*.
- Static modifier berlaku untuk *method*, dan variabel, meliputi *modifier* *static*.
- Abstract modifier berlaku untuk kelas, dan *method*, meliputi *modifier* *abstract*.
- Synchronized modifier berlaku untuk *method*, meliputi *modifier* *synchronized*.
- Native modifier berlaku untuk *method*, meliputi *modifier* *native*.
- Storage modifier berlaku untuk variabel, meliputi *modifier* *transient* dan *volatile*.

Deklarasi modifier dalam Java:

```
/*deklarasi modifier di class/interface*/  
[Modifier] class/interface [nama class/interface] {}  
  
//deklarasi modifier di atribut  
[Modifier][TypeData][nama Atribut];  
  
//deklarasi modifier di Method  
[Modifier][TypeData][nama Method]  
(parameter_1,parameter_2,parameter_n){}
```

Kesepakatan umum penamaan dalam suatu kelas:

- Nama Kelas : gunakan kata benda dan huruf pertama dari tiap kata ditulis dengan huruf besar dan memiliki access modifier *public*.
- Pada umumnya, atribut diberi access modifier *private*, dan method diberi access modifier *public*.
- Nama atribut : gunakan kata benda, dan diawali dengan huruf kecil : nama, umur
- Nama Method : gunakan kata kerja; kecuali huruf yang pertama, huruf awal tiap kata ditulis kapital : *getNama()*, *getUmur()*
- Untuk method yang akan memberikan atau mengubah nilai dari suatu atribut, nama method Kita tambahi dengan kata kunci "set": *setUmur(int a)*, *setNama(String a)*
- Untuk Method yang akan mengambil nilai dari atribut, nama Method Kita tambahi dengan kata kunci "get" : *getUmur()*, *getNama()*
- Konstanta. Semuanya ditulis dengan huruf besar; pemisah antar kata menggunakan garis bawah: *MAX_VALUE*, *DECIMAL_DIGIT_NUMBER*

2. Constructor

Constructor adalah tipe khusus method yang digunakan untuk menginstansiasi atau menciptakan sebuah objek. Nama *constructor* adalah sama dengan nama kelasnya. Selain itu, *constructor* tidak bisa mengembalikan suatu nilai bahkan *void* sekalipun. *Constructor* juga dimanfaatkan untuk membangun suatu objek dengan langsung mengeset atribut-atribut yang disandingkan pada objek yang dibuat tersebut.

Access *modifier* yang digunakan pada constructor selayaknya adalah *public* karena *constructor* tersebut akan diakses diluar kelasnya. Cara memanggil *constructor* adalah dengan menambahkan *keyword new*.

Deklarasi *constructor*;

```
[modifier] namaclass(parameter1){
    Body constructor;
}
[modifier] namaclass(parameter1,parameter2){
    Body constructor;
}
[modifier] namaclass(parameter1,parameter2,...,parameterN){
    Body constructor;
}
```

3. Package

Package digunakan untuk mengatur *class* yang telah dibuat dan akan bermanfaat jika *class* yang dibuat sangat banyak sehingga perlu dikelompokkan berdasarkan kategori tertentu. Selain pengelompokkan *class*, penggunaan *package* dapat mengatasi konflik penamaan *method* jika suatu *method* ditulis dalam *package* yang berbeda.

Terdapat dua cara menggunakan suatu *package*;

- Class* yang menggunakan berada dalam direktori (*package*) yang sama dengan *Class* yang digunakan, Maka tidak diperlukan *import*.
- Class* yang menggunakan berada dalam direktori (*package*) yang berbeda dengan *Class* yang digunakan, Maka perlu dilakukan *import package*.

B. Guided

1. Modifier & Constructor

```
package com.week4.modul.enkapsulasi.Guided.Guided1.manusia;

public class Manusia {
    //definisi atribut
    private String nama;
    private int umur;
```

```

//constructor
public Manusia(){};
public Manusia(String nama){
    this.nama = nama;
}

public Manusia(String nama, int umur){
    this.nama = nama;
    this.umur = umur;
}

//Method setter
public void setNama(String a) {
    nama = a;
}

public void setUmur(int umur) {
    this.umur = umur;
}

//Method getter
public String getNama() {
    return nama;
}

public int getUmur() {
    return umur;
}
}

```

```

package com.week4.modul.enkapsulasi.Guided.Guided1.main;

import com.week4.modul.enkapsulasi.Guided.Guided1.manusia.Manusia;
import guided.guided1.manusia.manusiaa;

public class DemoManusia {
    public static void main(String[] arg){

        Manusia arrMns[] = new Manusia[3];

        //constructor pertama
        Manusia objMns1 = new Manusia();

        //kedua
        Manusia objMns2 = new Manusia("Jhon");

        //ketiga
        Manusia objMns3 = new Manusia("Bajuri", 44);

        arrMns[0] = objMns1;
    }
}

```

```

arrMns[1] = objMns2;
arrMns[2] = objMns3;

for (int i = 0; i < 3; i++) {
    System.out.println("Nama: " + arrMns[i].getNama());
    System.out.println("Umur: " + arrMns[i].getUmur());
    System.out.println();
}
}
}

```

Output :

```

--- exec:3.6.3:exec (default-c11) @ modu
Nama: null
Umur: 0

Nama: Jhon
Umur: 0

Nama: Bajuri
Umur: 44

-----
BUILD SUCCESS
-----

Total time: 1.859 s
Finished at: 2026-10-06T21:39:30+07:00

```

2. Package

```

package com.week4.modul.enkapsulasi.Guided.Guided2.hargapulsa;

public class HargaPulsa {
    public void info() {
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}

```

```

package com.week4.modul.enkapsulasi.Guided.Guided2.hargatoken;

public class HargaToken {
    public void info() {
        System.out.println("Harga Token");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}

```

```
        System.out.println("Rp. 200000");
        System.out.println("Rp. 500000");
    }
}
```

```
package com.week4.modul.enkapsulasi.Guided.Guided2.main;

import com.week4.modul.enkapsulasi.Guided.Guided2.hargapulsa.HargaPulsa;
import com.week4.modul.enkapsulasi.Guided.Guided2.hargatoken.HargaToken;

public class Main {
    public static void main(String[] args) {
        HargaPulsa pulsa = new HargaPulsa();
        pulsa.info();

        HargaToken token = new HargaToken();
        token.info();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cil) @ modul-4-6
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000
Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000
-----
BUILD SUCCESS
-----
Total time:  1.783 s
Finished at: 2026-10-06T21:43:45+07:00
-----
```

C. Unguided

1. Soal Unguided 1

Sistem Pencatatan Dataset

Kamu sedang membangun program kecil untuk mencatat dan memeriksa kualitas dataset sebelum dianalisis, seperti tahap awal pengecekan *missing value*. Setiap

dataset punya nama, jumlah baris, jumlah kolom, dan jumlah sel yang kosong (*missing*).

Jawaban :

```
package com.week4.modul.enkapsulasi.Unguided.model;

public class Dataset {
    public static final double BATAS_MISSING = 5.0;
    private static int totalDataset = 0;

    private String nama;
    private int jumlahBaris;
    private int jumlahKolom;
    private int jumlahMissing;

    public Dataset(){
        this.nama = "";
        totalDataset++;
    }

    public Dataset(String nama){
        this.nama = nama;
        totalDataset++;
    }

    public Dataset(String nama, int jumlahBaris, int jumlahKolom, int
jumlahMissing){
        this(nama);
        setJumlahBaris(jumlahBaris);
        setJumlahKolom(jumlahKolom);
        setJumlahMissing(jumlahMissing);
    }

    public void setNama(String nama){
        this.nama = nama;
    }

    public void setJumlahBaris(int jumlahBaris) {
        if (jumlahBaris >= 0){
            this.jumlahBaris = jumlahBaris;
        }
    }

    public void setJumlahKolom(int jumlahKolom) {
        if (jumlahKolom >= 0){
            this.jumlahKolom = jumlahKolom;
        }
    }
}
```

```

    public void setJumlahMissing(int jumlahMissing) {
        if (jumlahMissing >= 0){
            this.jumlahMissing = jumlahMissing;
        }
    }

    public static int getTotalDataset(){
        return totalDataset;
    }

    // Getter
    public String getNama() {return nama;}
    public int getJumlahBaris() {return jumlahBaris;}
    public int getJumlahKolom () {return jumlahKolom;}
    public int getJumlahMissing() {return jumlahMissing;}

    // Persentase sel kosong terhadap total sel
    public double getPersentaseMissing(){
        int totalSel = jumlahBaris * jumlahKolom;
        if (totalSel == 0){
            return 0;
        }
        return (double) jumlahMissing / totalSel * 100;
    }

    public boolean perluDibersihkan(){
        return getPersentaseMissing() > BATAS_MISSING;
    }

}

```

```

package com.week4.modul.enkapsulasi.Unguided.laporan;

import com.week4.modul.enkapsulasi.Unguided.model.Dataset;

public class LaporanDataset {
    public static void cetak(Dataset d){
        System.out.println("=== Laporan Dataset ===");
        System.out.println("Nama      : " + d.getNama());
        System.out.println("Jumlah Baris : " + d.getJumlahBaris());
        System.out.println("Jumlah Kolom : " + d.getJumlahKolom());
        System.out.println("Missing     : " + d.getJumlahMissing() + " Sel (" +
String.format("%.2f", d.getPersentaseMissing()) + "%)");
        System.out.println("Status      : " + (d.perluDibersihkan() ? "Perlu
dibersihkan" : "Bersih"));
        System.out.println();
    }
}

```

```

package com.week4.modul.enkapsulasi.Unguided.main;

import com.week4.modul.enkapsulasi.Unguided.model.Dataset;
import com.week4.modul.enkapsulasi.Unguided.laporan.LaporanDataset;

public class Main {
    public static void main(String[] args){

        // Objek 1: constructor tanpa parameter, isi dengan setter
        Dataset d1 = new Dataset();
        d1.setNama("Titanic");
        d1.setJumlahBaris(891);
        d1.setJumlahKolom(12);
        d1.setJumlahMissing(866);

        // Objek 2: constructor 1 parameter
        Dataset d2 = new Dataset("Wine Quality");

        // Objek 3: constructor lengkap
        Dataset d3 = new Dataset("Iris", 150, 5, 0);

        // Simpan dalam array
        Dataset[] daftar = new Dataset[3];
        daftar[0] = d1;
        daftar[1] = d2;
        daftar[2] = d3;

        // Cetak laporan dengan perulangan
        for (int i = 0; i < daftar.length; i++){
            LaporanDataset.cetak(daftar[i]);
        }

        System.out.println("Total dataset dibuat : " + Dataset.getTotalDataset());
    }
}

```

Output :

```

I      EXEC-3.0.3.EXEC (default CUI) & modul 4 enkapsulasi
=== Laporan Dataset ===
Nama      : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing    : 866 Sel (8,10%)
Status     : Perlu dibersihkan

=== Laporan Dataset ===
Nama      : Wine Quality
Jumlah Baris : 0
Jumlah Kolom : 0
Missing    : 0 Sel (0,00%)
Status     : Bersih

=== Laporan Dataset ===
Nama      : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing    : 0 Sel (0,00%)
Status     : Bersih

Total dataset dibuat : 3
-----
BUILD SUCCESS
-----

Total time: 1.234 s
Finished at: 2026-10-06T23:01:53+07:00

```

Penjelasan :

1. Dataset.java (package model)

- Konstanta : BATAS_MISSING nilainya tidak bisa diubah, dan milik class bukan milik tiap objek.
- Variabel : totalDataset tidak dapat diubah dari luar karena *private*.
- Atribut (enkapsulasi) : nama, jumlahBaris, jumlahKolom, dan jumlahMissing bersifat *private* sehingga class lain tidak bisa mengaksesnya langsung. Akses hanya lewat getter dan setter.
- Tiga constructor : Dataset() = membuat objek kosong. Dataset(String nama) = mengisi nama saja. Dataset(nama, baris, kolom, missing) = mengisi semua atribut. Ketiganya berbeda dalam jumlah dan tipe parameter, sehingga Java otomatis memilih yang sesuai saat *new Dataset()* dipanggil. Pada constructor ketiga *this(...)* memanggil constructor lain di class yang sama.
- Setter : setJumlahBaris, setJumlahKolom, dan setJumlahMissing jika nilai negatif, penugasan dilewati dan nilai lama tetap.
- getTotalDataset() : method ini dipanggil lewat nama class *Dataset.getTotaldataset()* tanpa membuat objek.

- `getPersentaseMissing()` : jika `totalSel` 0, akan me-return 0 untuk menghindari pembagian dengan nol. Dikali 100 agar menjadi persen.
 - `perluDibersihkan()` : mengembalikan true jika persentase melebihi 5.0.
2. `LaporanDataset.java` (package `laporan`)
- Mengimport class `Dataset` dari package `model`.
 - `Cetak(Dataset d)` : menerima satu objek `Dataset` sebagai parameter. Karena static pemanggilannya `LaporanDataset.ceta(..)` tanpa membuat objek `LaporanDataset`. Isi method mengambil data lewat getter lalu mencetaknya.
3. `Main.java` (package `main`)
- Objek 1 memakai constructor tanpa parameter, kemudian data diisi satu per satu lewat setter `setNama()`.
 - Objek 2 memakai constructor satu parameter. Baris, kolom dan missing tidak diisi, sehingga nilai awalnya otomatis 0.
 - Objek 3 memakai constructor lengkap.
 - Array dan perulangan : array menyimpan tiga objek, lalu *for* memanggil cetak untuk tiap elemen. `Daftar.length` lebih aman dari pada angka 3 yang ditulis langsung.
 - Total di akhir : mengembalikan 3 karena tiga objek sudah dibuat.

D. Kesimpulan

Modifier mengatur hak akses dan perilaku class, method dan atribut. Dengan menjadikan atribut `private` dan method `public` (getter/setter), data pada class `dataset` tersembunyi dan hanya bisa diakses melalui method, sehingga enkapsulasi tercapai. Modifier `final` membuat `BATAS_MISSING` menjadi konstanta, sedangkan `static` membuat `totalDataset` menjadi milik class yang dibagi bersama semua objek.

Constructor digunakan untuk membuat objek sekaligus mengisi nilai awalnya. Class `Dataset` memiliki tiga constructor dengan parameter berbeda, dan Java memilih yang sesuai saat objek dibuat. Hasilnya objek dapat dibuat dengan cara yang fleksibel.